Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on laboratory work no. 6

Course: Operating Systems

Topic: Inside the xv6 File System

Done by:
st. gr. FAF-243, Ilie Covali

Checked by:
Reițman Patricia

Chișinău – 2026

Operating Systems · FCIM / FAF · UTM · 2026-2027

Lab 6 · Inside the xv6 File System

Metadata & Environment

- **Course:** Operating Systems, FCIM / FAF, UTM, Academic Year 2026-2027
- **Lab Work:** Lab 6 (Track B: Build Your Own Small OS, xv6)
- **Author:** Ilie Covali (st. gr. FAF-243)
- **Verified by:** Reițman Patricia
- **Host OS:** Linux (CachyOS x86_64, Kernel 7.2.9-2-cachyos)
- **Target Architecture:** RISC-V 64-bit (`riscv64-elf-gcc` 14.2.0)
- **Emulator:** QEMU System Emulator (`qemu-system-riscv64` 10.1.0)
- **Source Tree:** `/home/ilie/xv6-riscv`
- **Workspace:** `/home/ilie/Documents/SO/os-lab6` (`~/os-lab6`)

Purpose of the Work

1. Study the on-disk file system layout and directory representations in the xv6 RISC-V operating system kernel.
2. Implement a user-space file metadata inspection utility (`mystat.c`) by querying the kernel `stat()` system call to track inode links and file types.
3. Trace the kernel lifecycle of inodes and data block deallocation during `sys_unlink()` and `iput()`.
4. Analyze structural filesystem parameters in `kernel/fs.h` (`BSIZE`, `NDIRECT`, `NINDIRECT`, `DIRSIZ`) and compute theoretical file size and disk metadata boundaries.
5. Author a recursive directory tree traversal utility (`find.c`) with cycle avoidance and resource leak protection, extending it to resolve hard-linked inodes.

Part 1. Read What xv6 Shows You

1. Root Directory Listing and Directory Creation

We boot the compiled kernel under QEMU and inspect the initial root directory layout, followed by creating a subfolder `a`:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 37288
echo           2 4 36104
sleep          2 5 35776
forktest       2 6 17848
grep           2 7 40808
init           2 8 36456
kill           2 9 36016
ln             2 10 35808
ls             2 11 39344
mkdir          2 12 36064
rm             2 13 36056
sh             2 14 59624
stressfs       2 15 36896
usertests      2 16 204288
grind          2 17 51928
wc             2 18 38288
zombie         2 19 35344
logstress      2 20 38056
forphan        2 21 36808
dorphan        2 22 36272
sync           2 23 35448
```

```
args            2 24 35768
mystat          2 25 36952
find            2 26 39184
console         3 27 0
$ mkdir a
$ ls a
.               1 28 32
..              1 1 1024
```

Observe (Part 1 Answers)

1. **Meaning of file types 1, 2, and 3 (`kernel/stat.h`):**
   - Type 1 (`T_DIR`): Directory.
   - Type 2 (`T_FILE`): Regular file.
   - Type 3 (`T_DEVICE`): Device special file (such as the UART/console driver).
2. **Inode number of the root directory and why `.` and `..` match:**
   - The root directory inode is `1` (`#define ROOTINO 1` in `kernel/fs.h`).
   - In any directory, `.` references the current directory and `..` references the parent directory. Because root is the top of the hierarchy, it has no parent directory above it; therefore, both `.` and `..` reference inode 1.
3. **Size of an empty directory and directory entry layout:**
   - A directory entry in xv6 is defined by `struct dirent` (`kernel/fs.h`):
     ```
     struct dirent {
       ushort inum;
       char name[DIRSIZ];
     };
     ```
     With `ushort inum` (2 bytes) and `DIRSIZ` (14 bytes), each entry is exactly **16 bytes**.
   - The reported directory size is 32 bytes ($32/16 = 2$). The empty directory contains exactly two initial entries: `.` (pointing to itself, inode 28) and `..` (pointing to the root parent directory, inode 1).

Part 2. Write `mystat`

1. Implementation of `user/mystat.c`

We implement `mystat.c`, resolving file types to human-readable strings and invoking `stat()` on each command-line argument:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
  switch (t) {
  case T_DIR:
    return "directory";
  case T_FILE:
    return "file";
  case T_DEVICE:
    return "device";
  default:
    return "?";
  }
}

int
main(int argc, char *argv[])
{
  struct stat st;
```

```
  if (argc < 2) {
    fprintf(2, "usage: mystat <path>...\n");
    exit(1);
  }

  for (int i = 1; i < argc; i++) {
    if (stat(argv[i], &st) < 0) {
      fprintf(2, "mystat: cannot stat %s\n", argv[i]);
      continue;
    }
    printf("%s: %s, inode %d, links %d, size %d bytes\n",
           argv[i], typename(st.type), st.ino, st.nlink, (int)st.size);
  }

  exit(0);
}
```

The program is added to `UPROGS` in `Makefile`:

```
UPROGS=\
    ...
    $U/_args\
    $U/_mystat\
    $U/_find\
```

2. Observing Hard Links in Operation

We run `mystat` across files, create hard link `r2`, and remove `README`:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 2, size 1024 bytes
console: device, inode 27, links 1, size 0 bytes
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
$ cat r2 | wc
48 336 2441
```

Observe (Part 2 Answers)

1. **Output comparison after `ln` and after `rm`:**
   - After `ln README r2`: Both `README` and `r2` reference inode 2, with the link count increasing from 1 to 2. File size (2441 bytes) and type remain identical.
   - After `rm README`: `r2` still references inode 2 and its size remains 2441 bytes, but its link count decrements back to 1. The data is preserved, as verified by `cat r2 | wc` returning 48 lines, 336 words, and 2441 bytes.
2. **Kernel unlinking code and data block deallocation:**
   - In `kernel/sysfile.c`, the `sys_unlink()` system call decrements the link counter at line 244:
     ```
     ip->nlink--;
     iupdate(ip);
     ```
   - Data blocks are freed inside `iput()` (`kernel/fs.c`, lines 357–365) via `itrunc(ip)`:
     ```
     int last = (ip->ref == 1 && ip->valid && ip->nlink == 0);
     if (last) {
       acquiresleep(&ip->lock);
       itrunc(ip);
       ...
     }
     ```
     Deallocation occurs only when the on-disk link count reaches zero (`ip->nlink == 0`) **and** the active in-memory reference count drops to 1 (`ip->ref == 1`), meaning no process has the file open.

3. **On-disk inode structure (`struct dinode` in `kernel/fs.h`):** The xv6 inode structure contains the following 6 fields:
   - `short type;` – File type (`T_DIR`, `T_FILE`, `T_DEVICE`).
   - `short major;` – Major device number.
   - `short minor;` – Minor device number.
   - `short nlink;` – Count of directory entries pointing to this inode.
   - `uint size;` – File size in bytes.
   - `uint addrs[NDIRECT + 1];` – Array of block numbers (12 direct blocks + 1 indirect block address).

Part 3. Measure the Limits

1. Filesystem Constants Inspection

Inspecting filesystem parameters in `kernel/fs.h`:

```
❯ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define DIRSIZ" kernel/fs.h
5:#define BSIZE  1024 // block size
26:#define NDIRECT  12
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
```

2. Disk Layout Metadata during Image Build

Building `fs.img` reveals disk allocation parameters:

```
❯ make clean ; make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
```

3. Maximum Filename Length Test in QEMU

Creating a directory whose name exceeds `DIRSIZ`:

```
$ mkdir averyveryverylongname
$ ls
...
averyveryveryl 1 29 32
...
```

Observe (Part 3 Answers)

1. **Filename truncation in `ls`:** `ls` prints `averyveryveryl` (14 characters). This is governed by `#define DIRSIZ 14` in `kernel/fs.h`. In `struct dirent`, filenames are allocated a fixed 14-byte buffer without mandatory trailing NUL bytes, truncating any name longer than 14 characters.
2. **Maximum File Size Calculation:**
   - Direct blocks: 12 blocks (`NDIRECT`).
   - Indirect block capacity: The indirect block stores 32-bit (`sizeof(uint) = 4`) block numbers:

$$\text{NINDIRECT} = \frac{\text{BSIZE}}{\text{sizeof(uint)}} = \frac{1024}{4} = 256 \text{ blocks}$$

   - Total blocks per file:

$$\text{MAXFILE} = 12 + 256 = 268 \text{ blocks}$$

   - Maximum byte size:

$$268 \text{ blocks} \times 1024 \text{ bytes/block} = 274,432 \text{ bytes} \approx 268 \text{ KiB}$$

3. **Disk Block Accounting from `nmeta`:**
   - **Total disk blocks:** 2000 (`FSSIZE 2000`).
   - **Metadata blocks (47 blocks total):**

  - Boot block: 1 block (block 0).
  - Super block: 1 block (block 1).
  - Log area: 31 blocks (30 data log blocks + 1 log header block).
  - Inode blocks: 13 blocks (stores 200 inodes; $1024/64 = 16$ dinodes per block, $\lceil 200/16 \rceil = 13$).
  - Free bitmap: 1 block (manages up to $1024 \times 8 = 8192$ blocks).
- **Data blocks:** 1953 blocks ($2000 - 47 = 1953$).

Part 4. Write `find` & Stretch

1. Implementation of `user/find.c`

We implement `user/find.c` using recursive directory scanning. To avoid file descriptor exhaustion (xv6 permits at most 16 open files per process), open directories are closed immediately when leaving scope:

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

void
find(char *path, char *target, int print_inum)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "find: cannot open %s\n", path);
    return;
  }

  if (fstat(fd, &st) < 0) {
    fprintf(2, "find: cannot stat %s\n", path);
    close(fd);
    return;
  }

  if (st.type != T_DIR) {
    close(fd);
    return;
  }

  if (strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)) {
    fprintf(2, "find: path too long\n");
    close(fd);
    return;
  }

  strcpy(buf, path);
  p = buf + strlen(buf);
  *p++ = '/';

  while (read(fd, &de, sizeof(de)) == sizeof(de)) {
    if (de.inum == 0)
      continue;

    char name[DIRSIZ + 1];
    memmove(name, de.name, DIRSIZ);
    name[DIRSIZ] = 0;
```

```
    if (strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
      continue;

    strcpy(p, name);

    if (stat(buf, &st) < 0) {
      fprintf(2, "find: cannot stat %s\n", buf);
      continue;
    }

    if (strcmp(name, target) == 0) {
      if (print_inum)
        printf("%s %d\n", buf, st.ino);
      else
        printf("%s\n", buf);
    }

    if (st.type == T_DIR) {
      find(buf, target, print_inum);
    }
  }

  close(fd);
}

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: find <path> <name> [-n]\n");
    exit(1);
  }

  int print_inum = 0;
  if (argc >= 4 && strcmp(argv[3], "-n") == 0) {
    print_inum = 1;
  }

  find(argv[1], argv[2], print_inum);
  exit(0);
}
```

2. Verification on a Clean Directory Tree

We create nested test directories and files, link `r2` to `cat`, and execute `find`:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
$ find a b
a/b
```

3. Stretch: Inode Identification with `-n`

Testing the `-n` option to print corresponding inode numbers:

```
$ find . r2 -n
./a/b/r2 30
./a/r2 31
./r2 3
```

Analysis of the three `r2` targets: - `ls` confirms that `cat` occupies inode 3 (`cat 2 3 37288`). - `./r2` shares **inode 3**, identifying it as the hard link referencing the `cat` binary. - `./a/b/r2` (inode 30) and `./a/r2` (inode 31) are distinct files created via shell redirection.

Conclusions

This lab examined the architecture of the xv6 file system: 1. **Uniform Inode Model:** The xv6 filesystem stores files, directories, and hardware devices under a common `struct dinode` format. Directory entries are 16-byte pairs of 14-byte names and 16-bit inode numbers. 2. **Hard Link Mechanics:** Hard links create direct directory references to an existing inode. Physical blocks are reclaimed by `itrunc()` only when both the on-disk link counter reaches zero and all processes close their file descriptors. 3. **Capacity Constraints:** The combination of 12 direct blocks and a single 256-entry indirect block caps maximum file size at 268 KiB on a 1024-byte block filesystem. 4. **Recursive Traversal:** Authoring `find.c` demonstrated safe directory recursion in an environment with limited file descriptor tables, ensuring descriptors are closed promptly after scanning.